

Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 1: Tools and Programming

Dr. Aurélien Sallin (Lecturer) Ana Armendariz (Tutor)

17 September 2026

Plan for today

1. Install R and RStudio
2. The RStudio window, and R as a calculator
3. Folders, paths and an RStudio Project
4. Scripts
5. Objects, and the arrow `<-`
6. Packages
7. Quarto

Self-study: vectors

By the end you have a working installation, a course folder, your first script and the packages we need.

1. R and RStudio

R and RStudio are two different programs

R

The programming language and the engine that runs your code.

It does the computing.

Install R first, RStudio second.

Once installed, you open **RStudio**, not R.

RStudio

An IDE: editor, plot pane, help viewer, file browser.
It does no computing itself. It hands your code to R.

Installing R

<https://cran.r-project.org/>

Windows

Download R for Windows → *base* → the big link at the top. Accept every default.

macOS

Download R for macOS → the **arm64** .pkg for Apple silicon (M1 to M4), the **x86_64** .pkg for older Intel Macs. Accept every default.

Installing RStudio

<https://posit.co/download/rstudio-desktop/>

RStudio Desktop, the free version. Install with the defaults.

Exercise 1: get R running

1. Install R
2. Install RStudio
3. Open RStudio, click into the Console, type:

```
R.version.string
```

You should see "R version 4.4.1 ...". Any version 4 is fine.

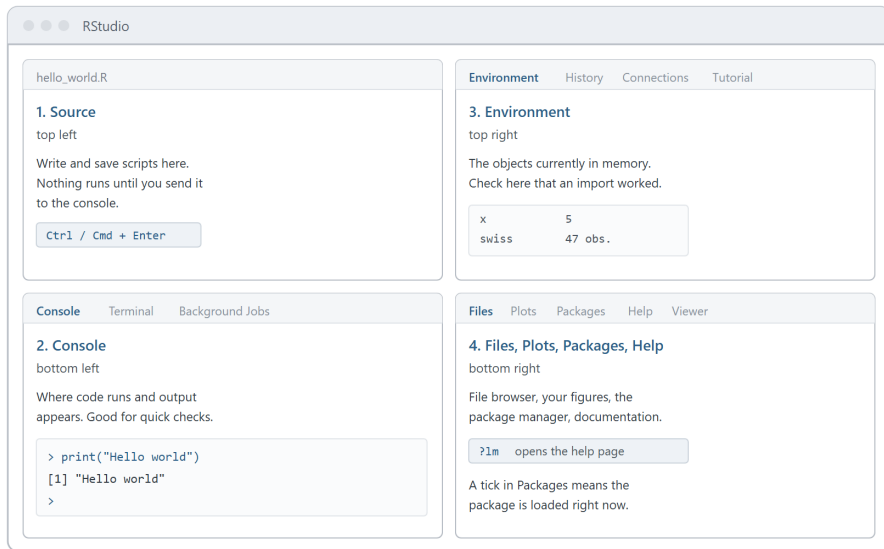
If anything fails, raise your hand.

Installation problems and how to fix them

Problem	Fix
RStudio cannot find R "R session failed to start"	You installed only RStudio. Install R, then restart RStudio. Then Tools > Global Options > General, point <i>R version</i> at your installation
macOS: "developer cannot be verified"	Right-click the file, <i>Open</i> . Or System Settings > Privacy & Security > Open Anyway
Windows user name with a space or accent	R struggles with the library path. Come and see me
No admin rights	Install "for this user only"

2. The RStudio window

RStudio has four panes



Source is for scripts, Console is where code runs

Source, top left

Where you write and save scripts.

Nothing here runs on its own.

Ctrl+Enter sends a line to R.

Environment, top right: the objects in memory.

Files / Plots / Packages / Help, bottom right.

Console, bottom left

Where code runs and output appears.

Nothing typed here is saved.

> means R is ready, + means your command is unfinished

> R is waiting for a command. Normal.

+ R thinks your command is unfinished, usually an open bracket or quotation mark.

Press Esc to cancel the line and type it again.

R as a calculator

```
2 + 2
```

```
[1] 4
```

```
20 / 8
```

```
[1] 2.5
```

```
2^3
```

```
[1] 8
```

```
sqrt(16)
```

```
[1] 4
```

```
round(3.14159, 2)
```

```
[1] 3.14
```

Arithmetic operators

Operator	Meaning	Example	Result
+	addition	7 + 3	10
-	subtraction	7 - 3	4
*	multiplication	7 * 3	21
/	division	7 / 3	2.333
^	power	7^2	49
%%	remainder	7 %% 3	1

Anything after # is a comment. R ignores it.

An error stops R, a warning does not

```
sqrt("four")
```

```
Error in `sqrt()`:
```

```
! non-numeric argument to mathematical function
```

R tells you what is wrong: `sqrt()` needs numbers, you gave it text.

An **error** means R stopped and did nothing.

A **warning** means R did something and wants you to know it was odd:

```
sqrt(-1)
```

```
Warning in sqrt(-1): NaNs produced
```

```
[1] NaN
```

? opens the help page for any command

```
?round      # the help page for round()  
??regression # search the documentation
```

Start with the **Examples** at the bottom of the page. Copy them into the Console and run them.

Exercise 2: the Console

Type each of these and press Enter:

1. `2 + 2`
2. `144 / 12`
3. `sqrt(81)`
4. `round(2.718281828, 3)`
5. `10 / 0`. Is this an error?
6. `sqrt(16` without the closing bracket. Look at the prompt, then press Esc
7. Open the help page for `round()`

3. Folders, paths and the working directory

Most file errors mean R is looking in the wrong folder

cannot open file almost always means R was looking in the wrong folder.

A **directory** is a folder. Directories form a tree:

Documents

 2026_Data_Handling

 course

 exercises

 data

 swiss.csv

 code

 hello_world.R

Absolute and relative paths

A **path** is the address of a file or a folder.

Absolute, starts at the top of your disk:

<code>C:/Users/anna/Documents/2026_Data_Handling/exercises/data/swiss.csv</code>	(Windows)
<code>/Users/anna/Documents/2026_Data_Handling/exercises/data/swiss.csv</code>	(Mac)

Relative, starts from where you are now:

<code>exercises/data/swiss.csv</code>	swiss.csv, in the folder data, inside exercises
<code>../swiss.csv</code>	swiss.csv, one folder up

Absolute paths break as soon as the folder moves. **Use relative paths.**

The working directory is where R looks

Where R currently is. Every relative path is read from it.

```
getwd()      # which folder am I in?  
list.files() # what is in it?
```

Run both whenever R says a file does not exist.

On Windows, write paths with **forward slashes**: C:/Users/...

A single backslash has a special meaning in R and will cause errors.

An RStudio Project sets the working directory for you

A folder with a small `.Rproj` file in it.

Open the project and RStudio sets the working directory to that folder automatically, on any computer.

Your relative paths always work, and you never need `setwd()`.

Exercise 3: your course folder

1. Create a folder `2026_Data_Handling`, for example in Documents
2. Inside it, create two subfolders, `course` and `exercises`
3. Inside `exercises`, create two subfolders, `data` and `code`
4. In RStudio: **Project** button, top right → **New Project...** > **Existing Directory** → browse to `2026_Data_Handling` → **Create Project**
5. Check with `getwd()`. The path should end in `/2026_Data_Handling`

`course` is for the lecture material, `exercises` for the exercise sessions.

4. Scripts

Use the Console to try things, a script to keep them

Everything you type in the Console disappears when you close RStudio.

A **script** is a plain text file, ending in `.R`, that holds your code so you can save it, correct it and run it again.

- ▶ `Ctrl+Enter` (`Cmd+Enter`) runs the current line or the selection
- ▶ the **Run** button does the same
- ▶ `source("exercises/code/my_script.R")` runs a whole script from the Console

Writing a line in the Source pane does nothing until you send it to R.

Exercise 4: your first script

1. In the Console: `print("Hello world")`
2. File > New File > R Script
3. Type `print("Hello world")` on the first line
4. Ctrl+Enter to run it
5. Save it into exercises/code as `hello_world.R`
6. In the Console:

```
source("exercises/code/hello_world.R")
```

The path in step 6 is relative, and it works because the project set your working directory.

5. Objects, and the arrow <-

Save a result with the arrow <-

```
x <- 5
```

Read it as “x gets 5”. Nothing prints: R stored the value instead of showing it.

x now appears in the **Environment** pane.

```
x
```

```
[1] 5
```

Objects can be used to build other objects

```
y <- 3  
z <- x + y  
z
```

[1] 8

z was computed once. Changing x afterwards does not update z:

```
x <- 100  
z
```

[1] 8

No live link between objects, unlike a spreadsheet.

Four rules for naming objects

- ▶ **R overwrites without warning.** `x <- 100` silently replaced the 5
- ▶ **Names are case sensitive.** `data`, `Data`, `DATA` are three objects
- ▶ **Names start with a letter**, no spaces, no accents. `income_2023`, not `2023_income`
- ▶ **Use `<-`, not `=`.** Shortcut: `Alt+-` (`Option+-` on Mac)

```
ls()           # what is in memory
rm(list = ls()) # clear it
```

Exercise 5: objects

1. Store your age in an object called age, and print it
2. Create `radius <- 3`, then area as πr^2 , rounded to two decimals. `pi` already exists in R
3. Type `Age`, with a capital A. What happens, and why?

6. Packages

A package is a set of commands written by other people

Collections of commands written by other people, published on CRAN.

Over 20,000 of them. In this course:

Package	For
dplyr	handling data
ggplot2	graphics
readxl	reading Excel files

`install.packages()` once, `library()` every session

```
install.packages("dplyr")    # ONCE per computer  
library(dplyr)               # EVERY time you open R
```

`install.packages()` downloads the package and copies it onto your hard disk. Quotation marks around the name.

`library()` makes an installed package available in this session. No quotation marks.

Install once, load every session.

Common package error messages

Message	Meaning
there is no package called 'dplyr'	Not installed. <code>install.packages("dplyr")</code>
could not find function "filter"	Installed but not loaded. <code>library(dplyr)</code>
built under R version ...	A warning, not an error. Usually harmless

If R asks about installing *from sources*, answer `n`.

The *masked from package:stats* message is information, not an error.

Exercise 6: packages

```
install.packages(c("dplyr", "ggplot2", "readxl"))  
  
library(dplyr)  
library(ggplot2)  
library(readxl)
```

3. In the Packages tab, bottom right, find dplyr. Is it ticked?
4. Restart R with Ctrl+Shift+F10, then run `ggplot()`. What happens, and what do you have to do about it?

7. Quarto

A Quarto document mixes text and runnable code

The `.qmd` files in the course repository, including your handout, are **Quarto** documents: normal writing plus executable R code chunks.

Rendering runs the code and drops the results, tables and plots straight into the finished PDF or HTML.

Nothing is copied by hand from R into Word, so the table and the data cannot disagree.

Try it: `File > New File > Quarto Document...`, accept the defaults, click **Render**.

Self-study: vectors

A vector holds several values at once

We come back to vectors in week 3. Read this at home if you want a head start.

Several values in one object, built with `c()` for “combine”:

```
income <- c(52000, 48000, 61000, 55000, 49000)
income
```

```
[1] 52000 48000 61000 55000 49000
```

Text goes in quotation marks:

```
canton <- c("SG", "ZH", "BE", "SG", "VD")
length(income)
```

```
[1] 5
```

```
mean(income)
```

```
[1] 53000
```

A column of a data set is exactly this.

Exercise 7: vectors

1. Create a vector `grades` holding 4.5, 5, 5.5, 4 and 6
2. How many values does it have? What is the average?

What you should have at the end of today

You should have:

- ▶ R and RStudio running
- ▶ a 2026_Data_Handling project with course and exercises inside
- ▶ hello_world.R saved and running
- ▶ dplyr, ggplot2 and readxl installed

Next week: importing and handling real data.

Questions?